Week1_예습과제

통계 수정 삭제

카사바 · 방금 전

0

Euron_2026-1

목록 보기 1/1

1장 머신 러닝과 딥러닝

1.1 인공지능, 머신 러닝과 딥러닝

인공지능: 인간의 지능을 모방하여 사람이 하는 일을 컴퓨터가 할 수 있도록 하는 기술

인공지능, 머신 러닝, 딥러닝의 관계는 다음과 같다:
인공지능 > 머신 러닝 > 딥러닝

머신 러닝: 주어진 데이터를 인간이 먼저 전처리한다. 범용적인 목적을 위해 제작된 것으로 데이터의 특징을 스스로 추출하지 못한다. 따라서 이 과정을 인간이 처리해 주어야 한다.

딥러닝: 인간이 하던 작업을 생략한다. 대량의 데이터를 신경망에 적용하면 컴퓨터가 스스로 분석한 후 답을 찾는다.

1.2 머신 러닝이란

1.2.1 머신 러닝 학습 과정

크게 학습 단계와 예측 단계로 구분할 수 있다.

학습 단계: 훈련 데이터를 머신 러닝 알고리즘에 적용하여 학습시킨다. 이 학습 결과로 모형이 생성된다.

예측 단계: 모형에 새로운 데이터를 적용하여 결과를 예측한다.

머신 러닝의 주요 구성 요소로는 데이터와 모델이 있다.

데이터: 머신 러닝이 학습 모델을 만드는 데 사용하는 것. 실제 데이터의 특징이 잘 반영되고 편향되지 않는 훈련 데이터를 확보하는 것이 중요하다. 데이터는 훈련용 데이터셋과 테스트용 데이터셋으로 분리해서 사용한다. 보통 데이터의 80%를 훈련용으로, 20%를 테스트용으로 분리하여 사용한다. 종종 훈련 데이터셋을 나누어 검증 데이터셋을 사용하는 경우도 있는데, 이를 통해 테스트 데이터셋에 대한 성능을 가늠할 수 있다. 그리고 모델 성능을 높이는 데에도 도움을 줄 수 있다. 만약 검증 데이터셋에 비해 훈련 데이터셋의 정확도가 더 높다면, 과대적합이 일어났을 가능성이 있다. 이 경우 정규화를 하거나 에포크를 줄임으로써 과적합을 막을 수 있다.

모델: 머신 러닝의 학습 단계에서 얻은 최종 결과물로 가설이라고도 한다. 모델 학습 절차는 다음과 같다:
1. 모델(또는 가설) 선택
2. 모델 학습 및 평가
3. 평가를 바탕으로 모델 업데이트

1.2.2 머신 러닝 학습 알고리즘

지도 학습, 비지도 학습, 강화 학습이 있다.

- 지도 학습: 정답이 무엇인지 컴퓨터에게 알려 주고 학습시키는 방법
- 비지도 학습: 정답을 알려 주지 않고 특징이 비슷한 데이터를 클러스터링하여 예측하는 학습 방법
- 강화 학습: 자신의 행동에 대한 보상을 받으며 학습을 진행한다. 게임이 대표적인 사례. 보상이 커지는 행동은 자주 하도록 하고, 줄어드는 행동은 덜 하도록 하여 학습을 진행한다.

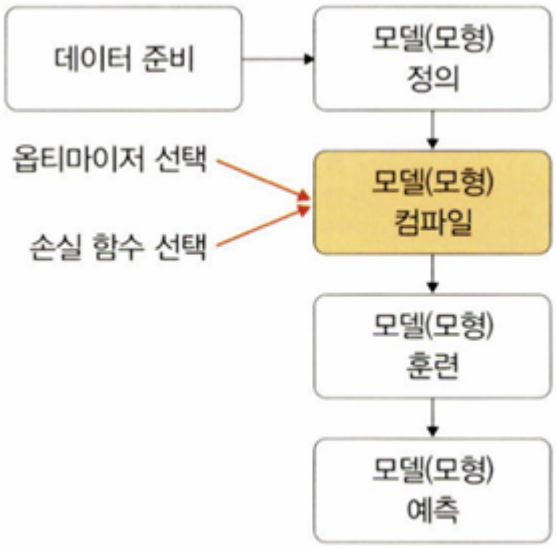
각 학습에서 자주 사용되는 알고리즘:

구분	유형	알고리즘
지도 학습 (supervised learning)	분류(classification)	• K-최근접 이웃(K-Nearest Neighbor, KNN) • 서포트 벡터 머신(Support Vector Machine, SVM) • 결정 트리(decision tree) • 로지스틱 회귀(logistic regression)
	회귀(regression)	선형 회귀(linear regression)
비지도 학습 (unsupervised learning)	군집(clustering)	• K-평균 군집화(K-means clustering) • 밀도 기반 군집 분석(DBSCAN)
	차원 축소 (dimensionality reduction)	주성분 분석 (Principal Component Analysis, PCA)
강화 학습 (reinforcement learning)	-	마르코프 결정 과정 (Markov Decision Process, MDP)

1.3 딥러닝이란

딥러닝: 인간의 신경망 원리를 모방한 심층 신경망 이론을 기반으로 고안된 머신 러닝 방법의 일종. 각각의 뉴런은 복잡하게 연결된 수많은 뉴런을 병렬 연산하여 기존에 컴퓨터가 수행하지 못했던 음성, 영상 인식 등의 처리를 가능하게 한다.

1.3.1 딥러닝 학습 과정

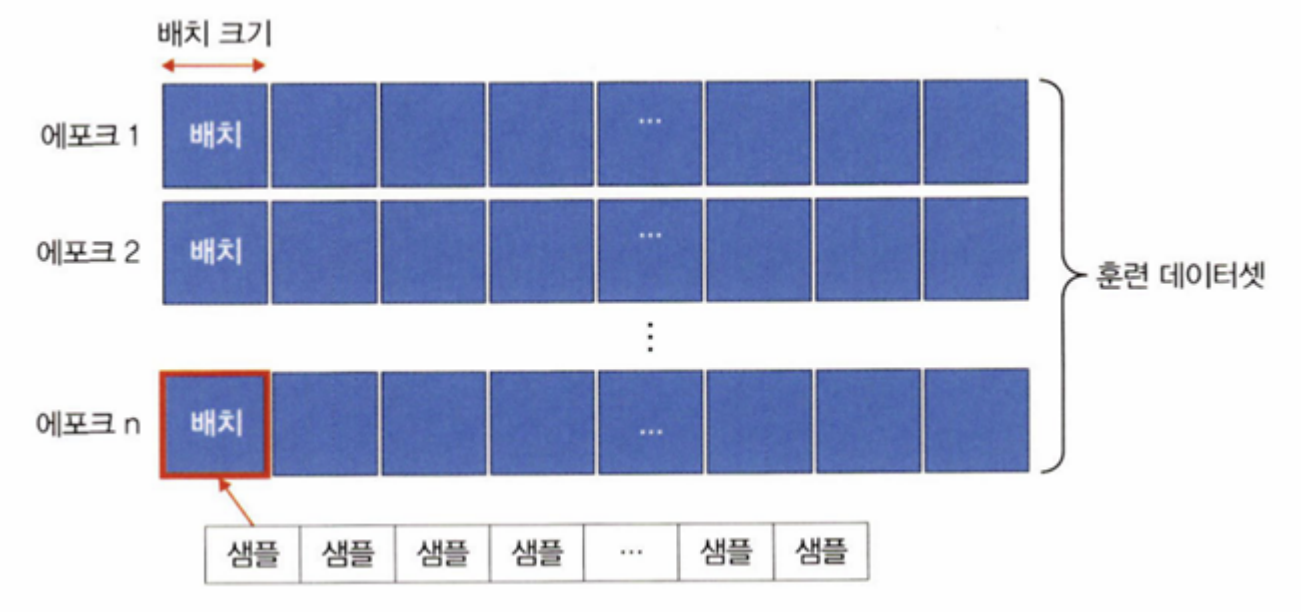


데이터 준비: 파이토치나 케라스에서 제공하는 데이터셋을 사용하거나 캐글 같은 곳에 공개된 데이터를 사용하는 것이 쉽다.

모델(모형) 정의: 신경망을 생성한다. 은닉층 개수가 많을수록 성능이 좋아지지만 과적합 발생 확률이 높아진다.

모델(모형) 컴파일: 활성화 함수, 손실 함수, 옵티마이저를 선택한다. 훈련 데이터셋 형태가 연속형이라면 평균 제곱 오차(MSE)를 사용할 수 있으며, 이진 분류라면 크로스 엔트로피를 선택한다. 그리고 과적합을 피할 수 있는 활성화 함수 및 옵티마이저 선택이 중요하다.

모델(모형) 훈련: 한 번에 처리할 데이터양을 지정한다. 한 번에 처리해야 할 데이터양이 많아지면 학습 속도가 느려지고 메모리 부족 문제를 야기할 수 있기 때문에 적당한 데이터양을 선택하는 것이 중요하다. 따라서 전체 훈련 데이터셋에서 일정한 묶음으로 나누어 처리할 수 있는 배치와 훈련의 횟수인 에포크 선택이 중요하다. 훈련 과정에서 값의 변화를 시각적으로 표현하여 눈으로 확인하면서 파라미터와 하이퍼파라미터에 대한 최적의 값을 찾을 수 있어야 한다.



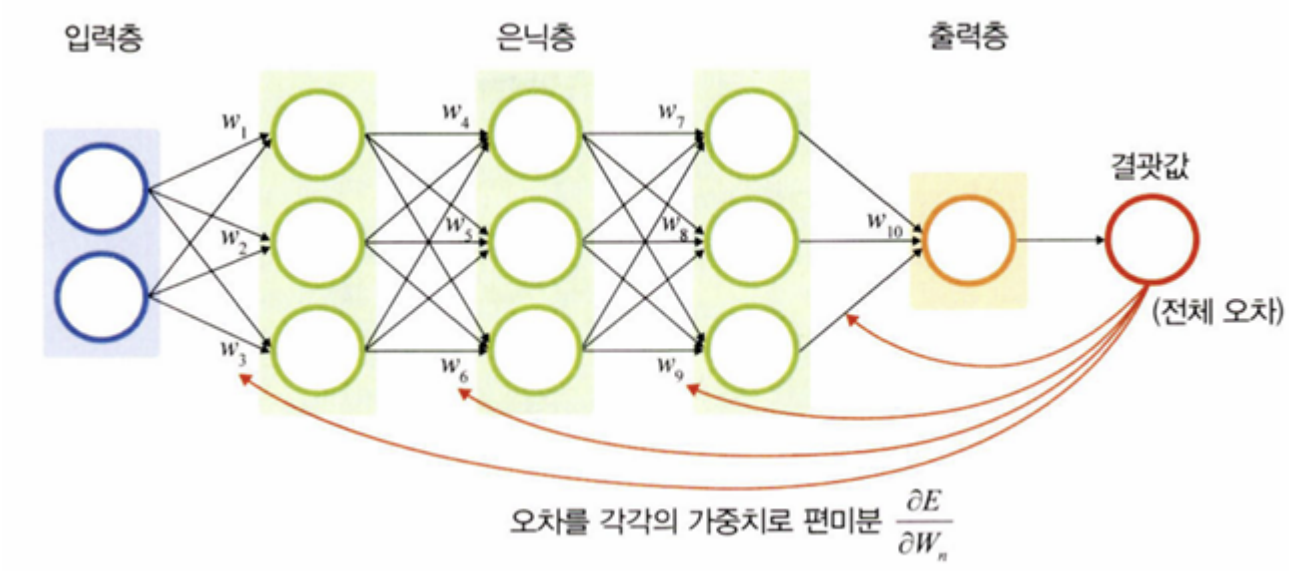
훈련 데이터셋 1000개에 대한 배치 크기가 20 :
샘플 단위 20개마다 모델 가중치를 한 번씩 업데이트시킨다. 즉, 총 50번(=1000/20)의 가중치가 업데이트된다. 이때 에포크가 10이고 배치 크기가 20이라면, 가중치를 50번 업데이트하는 것을 총 열 번 반복한다. 각 데이터 샘플이 총 열 번씩 사용되는 것이므로 결과적으로 가중치가 500번 업데이트된다.

모델(모형) 예측: 실제로 예측을 진행해 보는 단계. 예측력이 낮다면 파라미터를 튜닝하거나 신경망 자체를 재설계해야 한다.

딥러닝 학습 과정에서 중요한 핵심 구성 요소는 신경망과 역전파이다.

심층 신경망(은닉층이 두 개 이상인 신경망)에는 데이터셋의 어떤 특성들이 중요한지 스스로에게 가르쳐 줄 수 있는 기능이 있다.

가중치 값을 업데이트하기 위한 역전파가 중요하다. 특히 역전파 계산 과정에서 사용되는 미분(오차를 각 가중치로 미분)이 성능에 영향을 미치는 주요한 요소이다.



파이토치 같은 프레임워크를 이용하면 역전파 알고리즘을 자동으로 처리해 준다. 즉, 딥러닝 알고리즘 구현이 간단하고 편리하다.

1.3.2 딥러닝 학습 알고리즘

활용 분야에 따라 지도 학습, 비지도 학습, 전이 학습, 강화 학습으로 분류된다.

지도 학습

합성곱 신경망(CNN): 컴퓨터 비전에서 가장 많이 사용된다. 목적에 따라 이미지 분류, 이미지 인식, 이미지 분할로 분류할 수 있다. 이미지 분류는 이미지를 알고리즘에 입력하면 그 이미지가 어떤 클래스에 속하는지 알려 주기 때문에 이미지 데이터를 유사한 것끼리 분류할 때 사용한다. 이미지 인식은 사진을 분석하여 그 안에 있는 사물의 종류를 인식하는 것이다. 이미지 분할은 영상에서 사물이나 배경 등 객체 간 영역을 픽셀 단위로 구분하는 기술이다.

순환 신경망(RNN): 시계열 데이터(시간에 따른 데이터)를 분류할 때 사용된다. 역전파 과정에서 기울기 소멸 문제가 발생한다는 단점이 있다. 이러한 문제점을개선하고자 게이트를 세 개 추가한 것이 LSTM이다. 망각 게이트, 입력 게이트, 출력 게이트를 도입하여 기울기 소멸 문제를 해결했으며 현재 시계열 문제에서 가장 활발히 사용되고 있다.

비지도 학습

워드 임베딩: 단어를 벡터로 표현한다. 자연어 처리 분야의 일종. 단어 의미를 벡터화하는 워드투벡터(Word2Vec)와 글로브(GloVe)를 가장 많이 사용한다.

군집: 아무런 정보가 없는 상태에서 데이터를 분류하는 방법. 한 클러스터 안의 데이터는 매우 비슷하게 구성하고 다른 클러스터의 데이터와 구분되도록 나누는 것이 목표이다. 머신 러닝에서 군집화를 처리할 때 딥러닝과 함께 사용하면 모델 성능을 높일 수 있기 때문에 머신 러닝 단독으로 군집 알고리즘을 적용하기보다 딥러닝과 함께 사용하면 좋다.

전이 학습

사전에 학습이 완료된 모델(사전 학습 모델)을 가지고 우리가 원하는 학습에 미세 조정 기법을 이용하여 학습시키는 방법이다. 따라서 전이 학습에는 사전에 학습이 완료된 모델이 필요하며, 학습이 완료된 모델을 어떻게 활용하는지에 대한 접근 방법이 필요하다.

사전 학습 모델은 풀고자 하는 문제와 비슷하면서 많은 데이터로 이미 학습이 되어 있는 모델이다. VGG, Inception, MobileNet 같은 사전 학습 모델을 사용하면 효율적인 학습이 가능하다. 따라서 분석하려는 주제에 맞는 사전 학습 모델을 선택하고 활용해야 한다.

구분	유형	알고리즘
지도 학습(supervised learning)	이미지 분류	• CNN • AlexNet • ResNet
	시계열 데이터 분석	• RNN • LSTM
비지도 학습 (unsupervised learning)	군집 (clustering)	• 가우시안 혼합 모델(Gaussian Mixture Model, GMM) • 자기 조직화 지도(Self-Organizing Map, SOM)
	차원 축소	• 오토인코더(AutoEncoder) • 주성분 분석(PCA)
전이 학습(transfer learning)	전이 학습	• 버트(BERT) • MobileNetV2
강화 학습(reinforcement learning)	–	마르코프 결정 과정(MDP)

2장 실습 환경 설정과 파이토치 기초

2.1 파이토치 개요

파이토치: 딥러닝 프레임워크. 구현이 간결하고 빠르다.

2.1.1 파이토치 특징 및 강점

GPU에서 텐서 조작 및 동적 신경망 구축이 가능한 프레임워크

GPU(Graphics Processing Unit): 연산 속도를 빠르게 하는 역할을 한다. 딥러닝에서는 기울기를 계산할 때 미분을 쓰는데, GPU를 사용하면 빠른 계산이 가능하다. 내부적으로 CUDA, cuDNN이라는 API를 통해 GPU를 연산에 사용할 수 있다. 병렬 연산에서 GPU 속도는 CPU보다 훨씬 빠르다.

텐서: 파이토치의 데이터 형태이다. 텐서는 단일 데이터 형식으로 된 자료들의 다차원 행렬이다. 텐서는 간단한 명령어(변수 뒤에 .cuda()를 추가)를 사용해서 GPU로 연산을 수행하게 할 수 있다.

동적 신경망: 훈련을 반복할 때마다 네트워크 변경이 가능한 신경망. 예를 들어, 학습 중에 은닉층을 추가하거나 제거하는 등 모델의 네트워크 조작이 가능하다. 연산 그래프를 정의하는 것과 동시에 값도 초기화되는 'Define by Run' 방식을 사용한다. 따라서 연산 그래프와 연산을 분리해서 생각할 필요가 없기 때문에 코드를 이해하기 쉽다.

벡터, 행렬, 텐서

벡터: [1.0, 1.1, 1.2]처럼 숫자들의 리스트로, 1차원 배열 형태이다. 1차원 축 = axis 0

행렬: 행과 열로 표현되는 2차원 배열 형태. 가로줄은 행, 세로줄은 열이라고 한다. 2차원 축 = axis 1

텐서: 3차원 이상의 배열 형태. 3차원 축(채널) = axis 2

파이토치에서 텐서를 표현하기 위해서는 다음 코드와 같이 torch.tensor()를 사용한다.

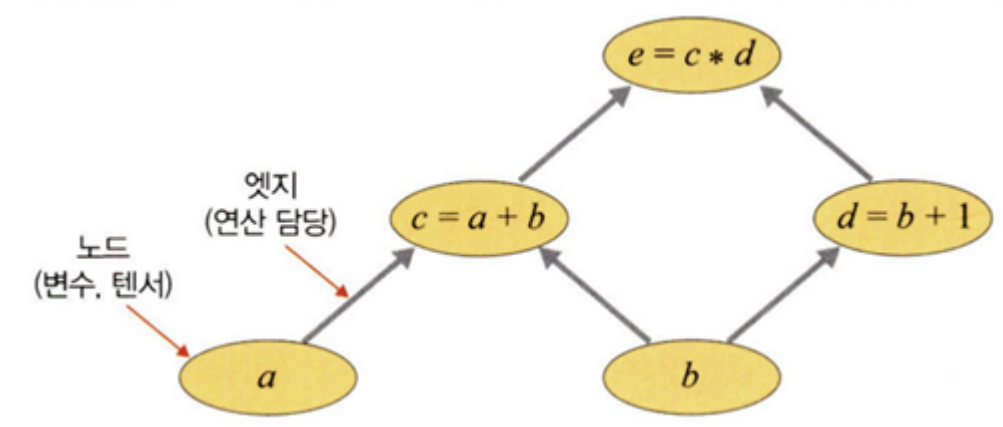
```
import torch
torch.tensor([[1., -1.], [1., -1.]])
```

생성된 텐서의 형태는 다음과 같이 표현된다.

```
torch.tensor([[1., -1.],
              [1., -1.]])
```

연산 그래프

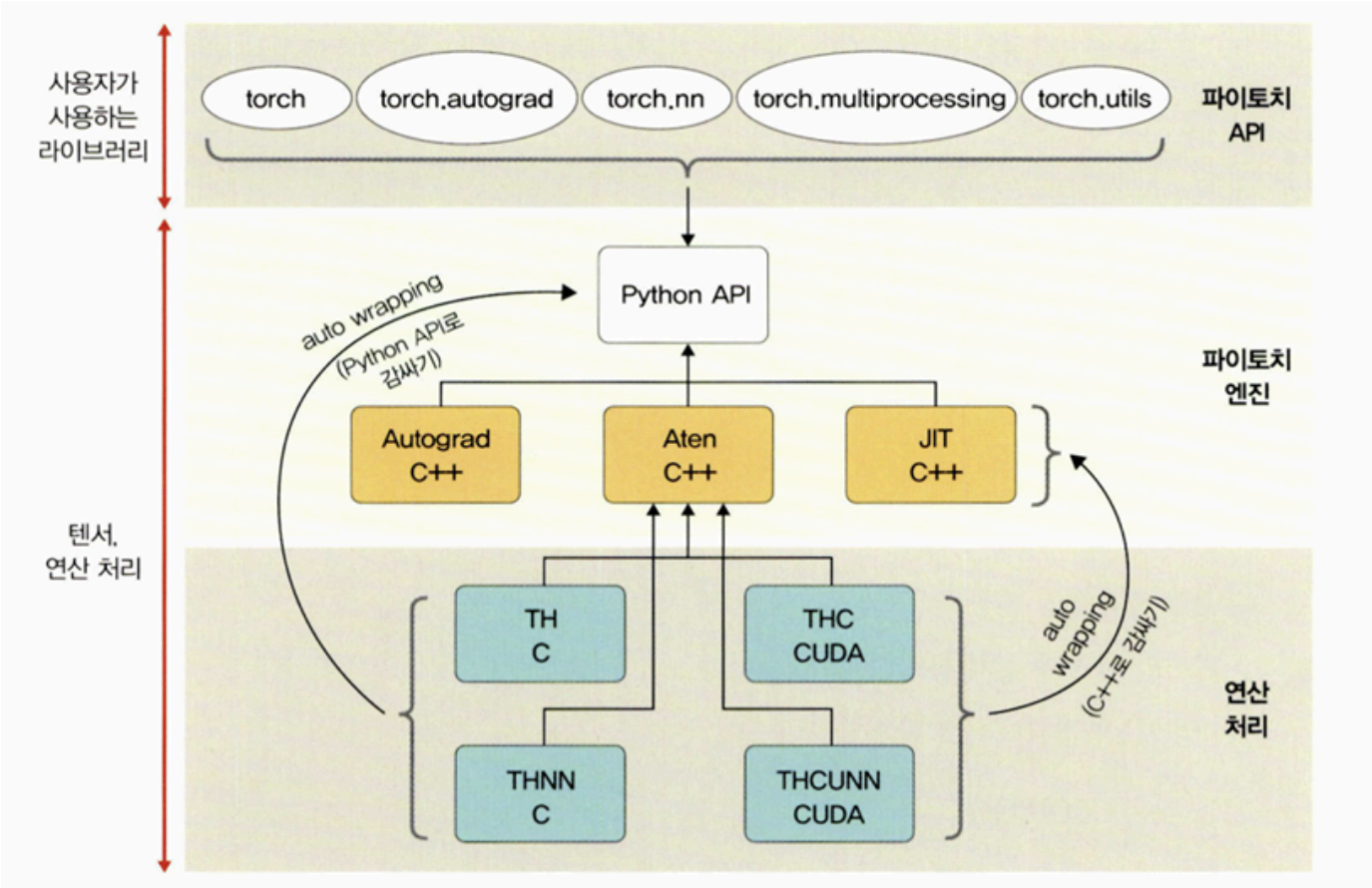
연산 그래프는 방향성이 있으며 변수(e.g. 텐서)를 의미하는 노드와 연산(e.g. 곱하기, 더하기)을 담당하는 엣지로 구성된다.



신경망은 연산 그래프를 이용하여 계산을 수행한다. 즉, 네트워크가 학습될 때 손실 함수의 기울기가 가중치와 바이어스를 기반으로 계산되며, 이후 경사 하강법을 사용하여 가중치가 업데이트된다. 이때 연산 그래프를 이용하여 이 과정이 효과적으로 수행된다.

2.1.2 파이토치의 아키텍처

파이토치 API, 파이토치 엔진, 연산 처리의 세 계층으로 나누어 설명할 수 있다.



파이토치 API

사용자가 이해하기 쉬운 API를 제공하여 텐서에 대한 처리와 신경망을 구축하고 훈련할 수 있도록 돕는다. 사용자 인터페이스를 제공하지만 실제 계산은 수행하지 않는다. 그 대신 C++로 작성된 파이토치 엔진으로 작업을 전달하는 역할만 한다. 사용자의 편의성을 위해 다음 패키지들이 제공된다.

- torch: GPU를 지원하는 텐서 패키지. 다차원 텐서를 기반으로 다양한 수학적 연산이 가능하도록 한다. 빠른 속도로 많은 양의 계산을 할 수 있다.
- torch.autograd: 자동 미분 패키지. 자동 미분이라는 기술을 채택하기 때문에 미분 계산을 효율적으로 처리한다. 즉, 연산 그래프가 즉시 계산되기 때문에 사용자는 다양한 신경망을 적용해 볼 수 있다.
- torch.nn: 신경망 구축 및 훈련 패키지. 합성곱 신경망, 순환 신경망, 정규화 등이 포함되어 손쉽게 신경망을 구축하고 학습시킬 수 있다.
- torch.multiprocessing: 파이썬 멀티프로세싱 패키지. 파이토치에서 사용하는 프로세스 전반에 걸쳐 텐서의 메모리 공유가 가능하다. 따라서 서로 다른 프로세스에서 동일한 데이터(텐서)에 대한 접근 및 사용이 가능하다.
- torch.utils: DataLoader 및 기타 유틸리티를 제공하는 패키지. 모델에 데이터를 제공하기 위한 torch.utils.data.DataLoader 모듈을 주로 사용한다. 또한 병목 현상을 디버깅하기 위한 torch.utils.bottleneck, 모델 또는 모델의 일부를 검사하기 위한 torch.utils.checkpoint 등의 모듈도 있다.

파이토치 엔진

Autograd C++, Aten C++, JIT C++, Python API로 구성되어 있다.

- Autograd C++: 가중치, 바이어스를 업데이트 하는 과정에서 필요한 미분을 자동으로 계산해 준다.
- Aten C++: 텐서 라이브러리 제공
- JIT C++: 계산을 최적화하기 위한 JIT 컴파일러이다.

연산 처리

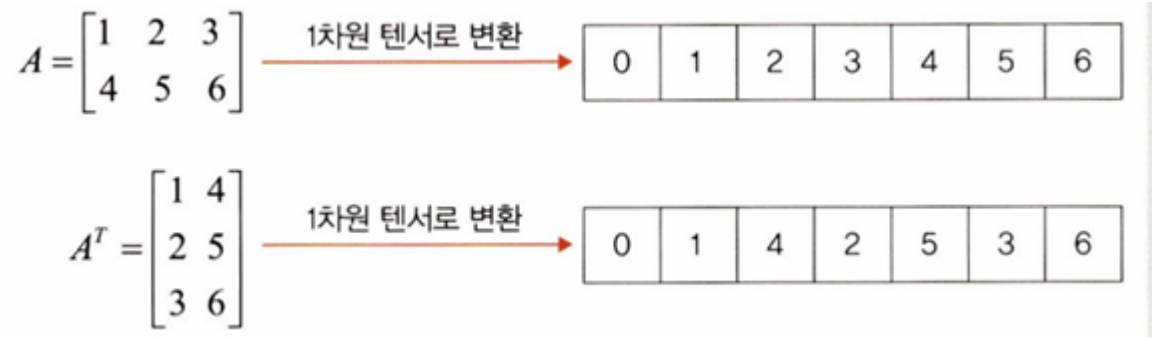
가장 아래 계층에 속하는 C 또는 CUDA 패키지는 상위의 API에서 할당된 거의 모든 계산을 수행한다.

텐서를 메모리에 저장하기

텐서는 1차원 배열 형태여야만 메모리에 저장할 수 있다. 변환된 1차원 배열을 스토리지라고 한다.

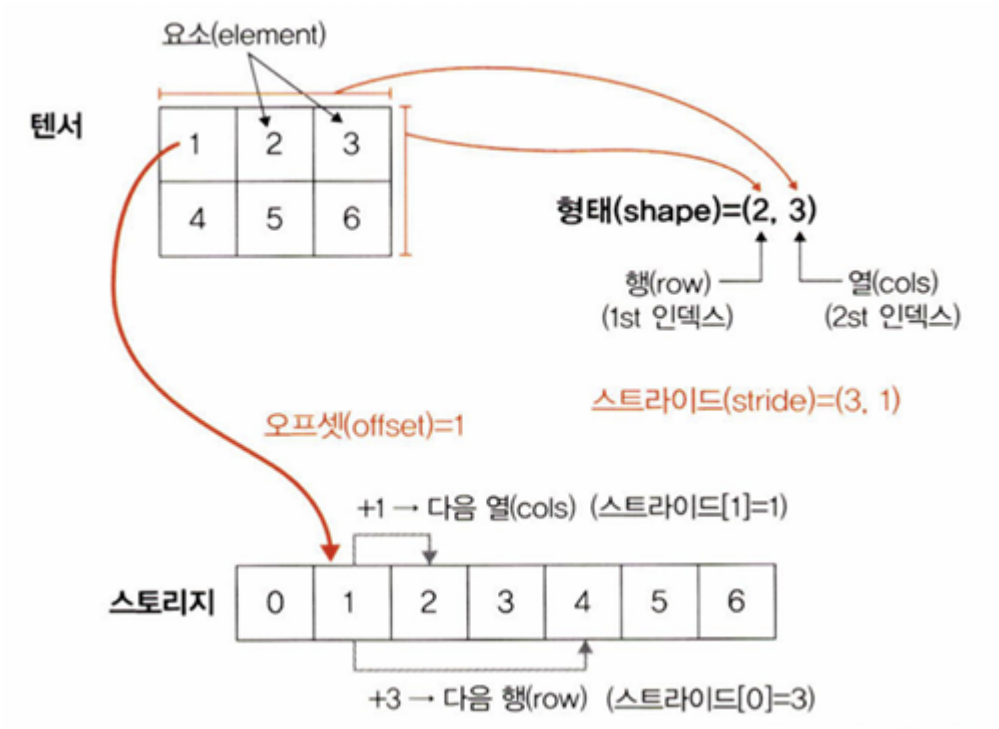
오프셋: 텐서에서 첫 번째 요소가 스토리지에 저장된 인덱스

스트라이드: 각 차원에 따라 다음 요소를 얻기 위해 건너뛰기가 필요한 스토리지의 요소 개수. 요소가 연속적으로 저장되기 때문에 행 중심으로 스트라이드는 항상 1이다.



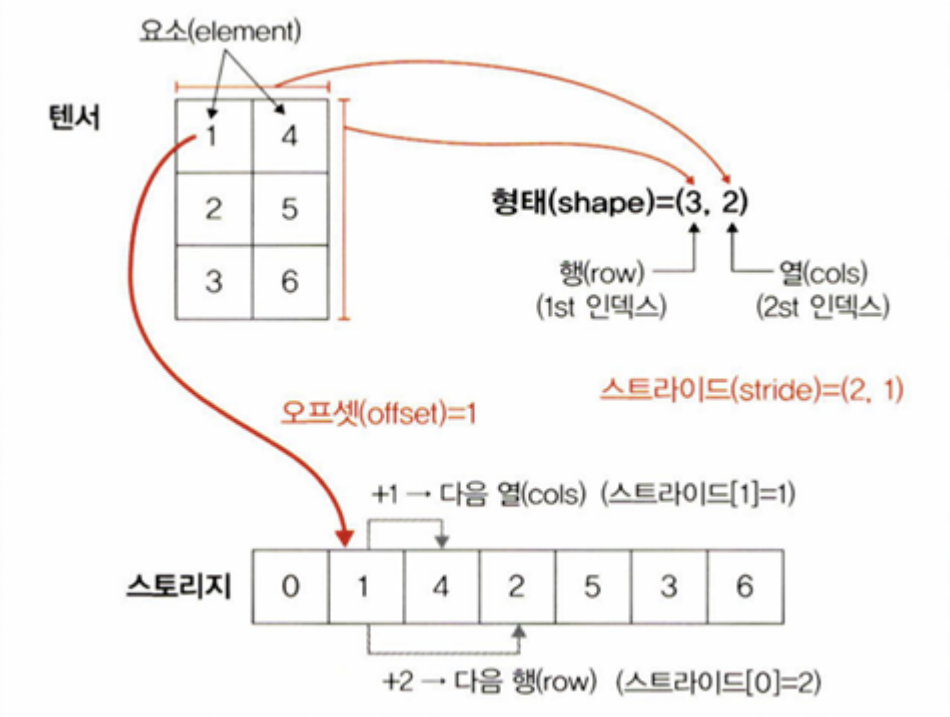
A와 A의 전치행렬은 다른 형태(shape)를 갖지만 스토리지의 값들은 서로 같다. 따라서 A와 전치행렬을 구분하는 용도로 오프셋과 스트라이드를 사용한다.

A



열을 기준으로 다음 요소를 얻기 위해서는 3을 뛰어넘어야 하고, 행을 기준으로 다음 요소를 얻기 위해서는 1을 뛰어넘어야 하므로 스토리지의 스트라이드는 (3,1)이다.

A의 전치행렬



열을 기준으로 다음 요소를 얻기 위해서는 2를 뛰어넘어야 하고, 행을 기준으로 다음 요소를 얻기 위해서는 1을 뛰어넘어야 하므로 스토리지의 스트라이드는 (2,1)이다.

이와 같이 오프셋과 스트라이드는 데이터 자체가 아닌 행렬/텐서를 구분하기 위해 사용한다.

2.2 파이토치 기초 문법

2.2.1 텐서 다루기

텐서의 인덱스 조작

텐서는 넘파이의 ndarray를 조작하는 것과 유사하게 동작하기 때문에 배열처럼 인덱스를 바로 지정하거나 슬라이스 등을 사용할 수 있다. 텐서의 자료형은 다음과 같다.

- torch.FlostTensor: 32비트의 부동 소수점
- torch.DoubleTensor: 64비트의 부동 소수점
- torch.LongTensor: 64비트의 부호가 있는 정수

코드에서 -1번 인덱스는 맨 뒤에서부터 시작하는 인덱스를 의미한다.

텐서 연산 및 차원 조작

텐서는 다양한 수학 연산이 가능하며, GPU를 사용하면 더 빠르게 연산할 수 있다. 텐서 간의

타입이 다르면 연산이 불가능하다.

텐서의 차원을 변경하는 가장 대표적인 방법은 view를 이용하는 것이다. 이외에도 텐서를 결합하는 stack, cat과 차원을 교환하는 t, transpose도 사용된다. view는 넘파이의 reshape과 유사하며 cat은 다른 길이의 텐서를 하나로 병합할 때 사용한다. transpose는 행렬의 전치 외에도 차원의 순서를 변경할 때도 사용된다.

2.2.2 데이터 준비

파이썬 라이브러리인 판다스를 이용하는 방법과 파이토치에서 제공하는 데이터를 이용하는 방법이 있다. 데이터가 이미지일 경우 분산된 파일에서 데이터를 읽은 후 전처리를 하고 배치 단위로 분할하여 처리한다. 데이터가 텍스트일 경우 임베딩 과정을 거쳐 서로 다른 길이의 시퀀스를 배치 단위로 분할하여 처리한다.

단순하게 파일을 불러와서 사용

판다스 라이브러리를 사용하여 JSON, PDF, CSV 등의 파일을 불러오는 방법이다. 데이터가 복잡하지 않은 형태일 때 유용하다.

커스텀 데이터셋을 만들어서 사용

데이터를 한 번에 메모리에 불러와서 훈련시키는 것은 시간과 비용 측면에서 비효율적이다. 따라서 데이터를 한 번에 다 부르지 않고 조금씩 나누어 불러서 사용하는 방식이 커스텀 데이터셋이다.

torch.utils.data.DataLoader 객체는 학습에 사용될 데이터 전체를 보관했다가 모델 학습을 할 때 배치 크기만큼 데이터를 꺼내서 사용한다.

파이토치에서 제공하는 데이터셋 사용

토치비전은 파이토치에서 제공하는 데이터셋들이 모여 있는 패키지이다. MNIST, ImageNet 등을 제공한다.

https://pytorch.org/vision/0.8/datasets.html
위 링크에서 파이토치에서 제공하는 데이터셋을 확인할 수 있다.

2.2.3 모델 정의

모델을 정의하기 위해서는 모듈을 상속한 클래스를 사용한다.

- 계층(layer): 모듈 또는 모듈을 구성하는 한 개의 계층으로 합성곱층, 선형계층 등이 있다.
- 모듈(module): 한 개 이상의 계층이 모여서 구성된 것으로, 모듈이 모여 새로운 모듈을 만들 수도 있다.
- 모델(model): 최종적으로 원하는 네트워크로, 한 개의 모듈이 모델이 될 수도 있다.

`단순 신경망을 정의하는 방법`

매우 단순한 모델을 만들 때 사용한다.

```
model = nn.Linear(in_features=1, out_features=1, bias=True)
```

`nn.Module()을 상속하여 정의하는 방법`

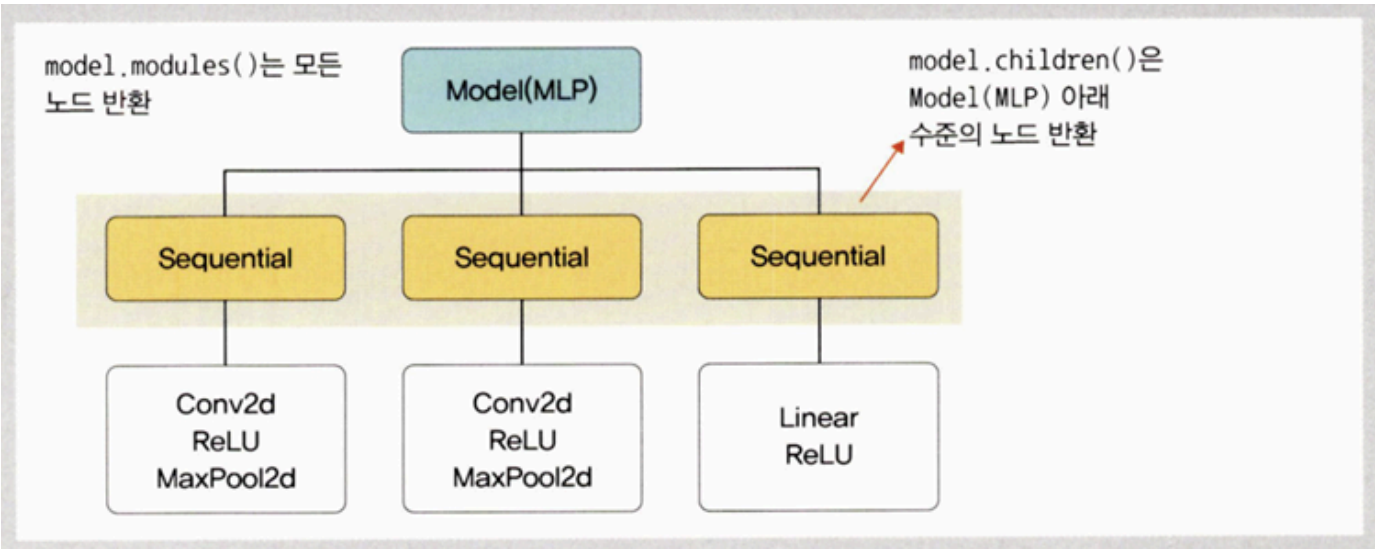
nn.Module()을 상속받는 모델은 기본적으로 `__init__()` 과 forward() 함수를 포함한다. `__init__()` 에서는 모델에서 사용될 모듈(nn.Linear, nn.Conv2d), 활성화 함수 등을 정의하고, forward() 함수에서는 모델에서 실행되어야 하는 연산을 정의한다.

`Sequential 신경망을 정의하는 방법`

Sequential 객체는 그 안에 포함된 각 모듈을 순차적으로 실행해 준다.

nn.Sequential은 모델의 계층이 복잡할수록 효과가 뛰어나다.

model.modules()는 모델의 네트워크에 대한 모든 노드를 반환하며, model.children()은 같은 수준(level)의 하위 노드를 반환한다.



`함수로 신경망을 정의하는 방법`

함수로 선언할 경우 변수에 저장해 놓은 계층들을 재사용할 수 있는 장점이 있다. 하지만 모델이 복잡해지는 단점도 있다. 복잡한 모델의 경우에는 함수를 이용하는 것보다는 nn.Module()을 상속받아 사용하는 것이 편리하다.

2.2.4 모델의 파라미터 정의

`손실 함수(loss function)` : 학습하는 동안 출력과 실제 값(정답) 사이의 오차를 측정한다. 즉, wx+b를 계산한 값과 실제 값인 y의 오차를 구해서 모델의 정확성을 측정한다. 손실 함수로 많이 사용되는 것은 다음과 같다:

- BCELoss: 이진 분류에 사용
- CrossEntropyLoss: 다중 클래스 분류에 사용
- MSELoss: 회귀 모델에 사용

`옵티마이저(optimizer)` : 데이터와 손실 함수를 바탕으로 모델의 업데이트 방법을 결정한다.

- 옵티마이저는 step() 메서드를 통해 전달받은 파라미터를 업데이트한다.
- 모델의 파라미터별로 다른 기준(e.g. 학습률)을 적용시킬 수 있다.
- torch.optim.Optimizer(params, defaults)는 모든 옵티마이저의 기본이 되는 클래스이다.
- zero_grad() 메서드는 옵티마이저에 사용된 파라미터들의 기울기(gradient)를 0으로 만든다.
- torch.optim.lr_scheduler는 에포크에 따라 학습률을 조절할 수 있다.

`학습률 스케줄러(learning rate scheduler)` : 미리 지정한 횟수의 에포크를 지날 때마다 학습률을 감소시켜 준다. 학습 초기에는 빠른 학습을 진행하다가 전역 최소점 근처에 다다르면 학습률을 줄여서 최적점을 찾아갈 수 있도록 해 준다.

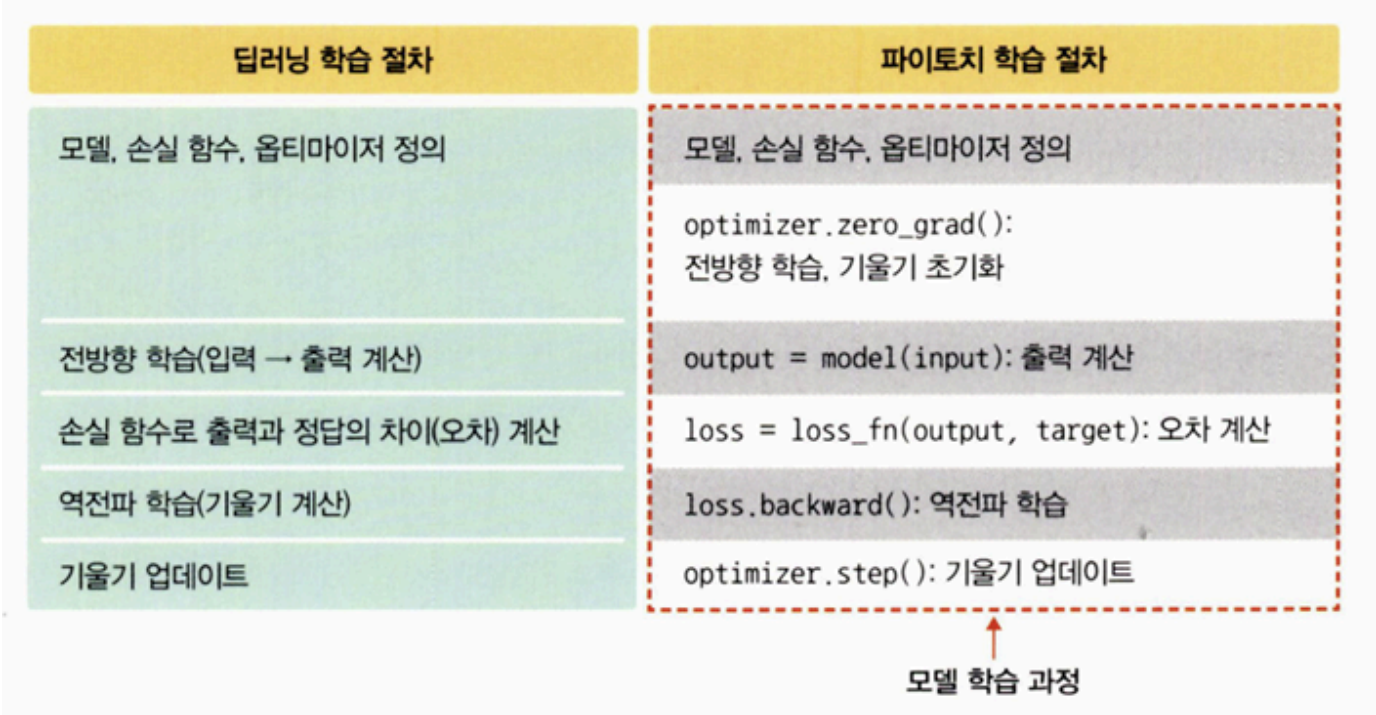
`지표(metrics)` : 훈련과 테스트 단계를 모니터링한다.

2.2.5 모델 훈련

학습을 시키는 것은 y=wx+b 함수에서 w와 b의 적절한 값을 찾는다는 의미이다. 오차가 줄어들어 전역 최소점에 이를 때까지 파라미터(w,b)를 계속 수정한다.

`훈련 방법`

optimizer.zero_grad() 메서드를 이용하여 기울기 초기화하기. 순환 신경망 모델을 구현할 때가 아니라면 이 메서드를 호출해서 미분 값이 누적되지 않게 초기화해 주어야 한다.



loss.backward() 메서드를 이용하여 기울기를 자동 계산한다. loss.backward()는 배치가 반복될 때마다 오차가 중첩적으로 쌓이게 되므로 매번 zero_grad()를 사용하여 미분 값을 0으로 초기화한다.

2.2.6 모델 평가

주어진 테스트 데이터셋을 사용하여 모델을 평가한다.

2.2.7 훈련 과정 모니터링

텐서보드를 이용하면 학습에 사용되는 각종 파라미터 값이 어떻게 변화하는지 시각화하여 볼 수 있다.

텐서보드 사용하는 방법:
1. 텐서보드 설정
2. 텐서보드에 기록
3. 텐서보드를 사용하여 모델 구조 살펴보기

```
model.train() & model.eval()
```
model.train(): 훈련 데이터셋에 사용하며 모델 훈련이 진행될 것임을 알린다. 이때 드롭아웃이 활성화된다.

model.eval(): 모델을 평가할 때 모든 노드를 사용하겠다는 의미로 검증과 테스트 데이터셋에 사용한다.

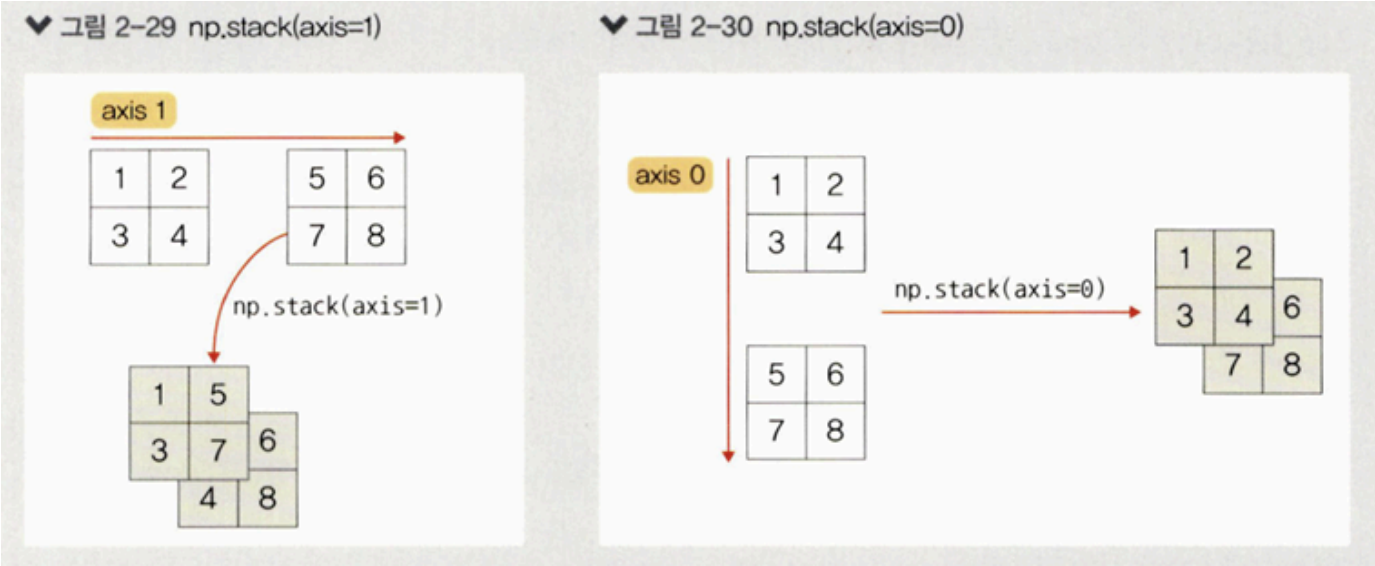
2.4 파이토치 코드 맛보기

범주형 데이터를 텐서로 변환하기 위해 다음과 같은 절차가 필요하다.

```
범주형 데이터 -> dataset[category] -> 넘파이 배열(NumPy array) -> 텐서(Tensor)
```

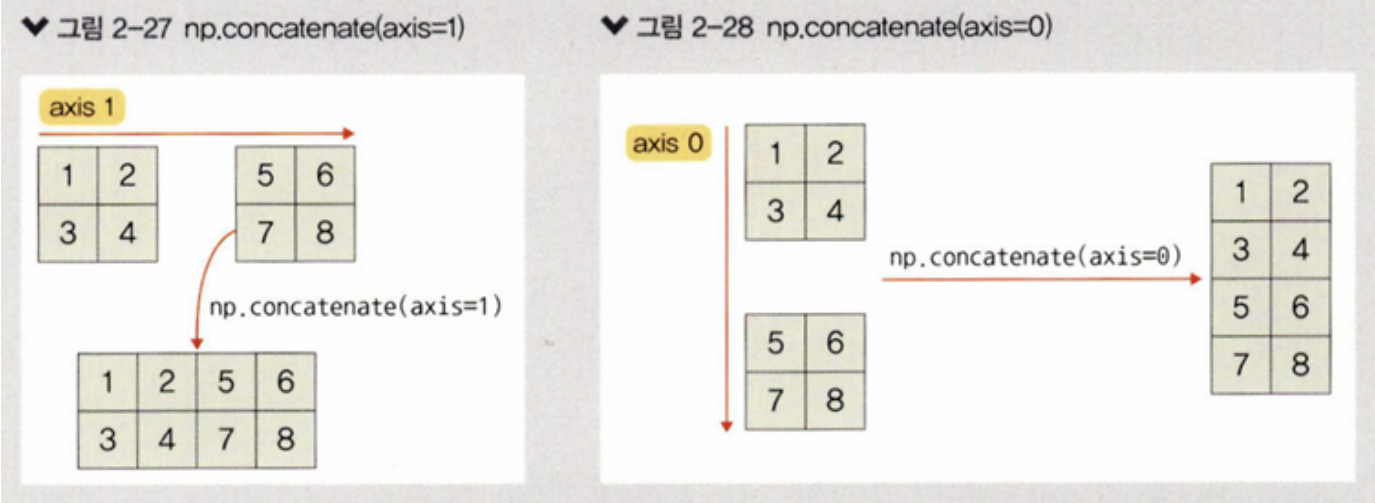
범주형 데이터(단어)를 숫자(넘파이 배열)로 변환하기 위해 cat.codes를사용한다.

np.stack은 두 개 이상의 넘파이 객체를 합칠 때 사용한다. 1차원 배열들을 합쳐서 2차원 배열을 만들거나 2차원 배열 여러 개를 합쳐 3차원 배열을 만든다. 따라서 만드시 두 배열의 차원이 동일해야 한다.



✔ 그림 2-29 np.stack(axis=1)
✔ 그림 2-30 np.stack(axis=0)

```
np.concatenate
```



✔ 그림 2-27 np.concatenate(axis=1)
✔ 그림 2-28 np.concatenate(axis=0)

get_dummies는 가변수(dummy variable)로 만들어 주는 함수이다. 이는 문자를 숫자 (0, 1)로 바꾸어 준다는 의미이다.

ravel(), reshape(), flatten()은 텐서의 차원을 바꿀 때 사용한다. 이 메서드들은 다음과 같이 사용할 수 있다.

```
a = np.array([[1, 2],
              [3, 4]])
print(a.ravel())
print(a.reshape(-1))
print(a.flatten())
```

코드를 실행하면 2차원 텐서가 1차원으로 변경되어 출력된다.

```
[1 2 3 4]
```

워드 임베딩: 유사한 단어끼리 유사하게 인코딩되도록 표현하는 방법

배열을 N차원으로 변환하기 위해 먼저 모든 범주형 칼럼에 대한 임베딩 크기(벡터 차원)를 정의한다. 임베딩 크기로 칼럼의 고유 값 수를 2로 나누는 것을 많이 사용한다. 예를 들어 price 칼럼은 네 개의 고유 값을 갖기 때문에 임베딩 크기는 4/2=2이다.

클래스 형태로 구현되는 모델은 nn.Module을 상속받는다.

`__init__()` 은 모델에서 사용될 파라미터와 신경망을 초기화하기 위한 용도로 사용하며, 객체가 생성될 때 자동으로 호출된다. 이 함수에 전달되는 매개변수는 다음과 같다.

- self: 자기 자신을 의미한다.
- embedding_size: 범주형 칼럼의 임베딩 크기
- output_size: 출력층의 크기
- layers: 모든 계층에 대한 목록
- p: 드롭아웃(기본값은 0.5)

`super().__init__()` 은 부모 클래스(Model 클래스)에 접근할 때 사용한다.

모델의 네트워크 계층을 구축하기 위해 for 문을 이용하여 각 계층을 all_layers 목록에 추가한다. 추가된 계층은 다음과 같다:

- Linear: 선형 계층은 입력 데이터에 선형 변환을 진행한 결과이다. 선형 변환을 위해서 다음 수식을 사용한다.
  y = Wx + b (y: 출력 값, W: 가중치, x: 입력 값, b: 바이어스)
- ReLU: 활성화 함수

- BatchNorm1d: 배치 정규화
- Dropout: 과적합 방지

forward() 함수는 학습 데이터를 입력받아서 연산을 진행한다.

성능 평가 지표로 정확도, 재현율, 정밀도, F1-스코어가 있다.

- True Positive: 모델이 1이라고 예측했고, 실제 값도 1인 경우
- True Negative: 모델이 0이라고 예측했고, 실제 값도 0인 경우
- False Positive: 모델이 1이라고 예측했는데 실제 값은 0인 경우
- False Negative: 모델이 0이라고 예측했는데 실제 값은 1인 경우

정확도: 전체 예측 건수에서 정답을 맞힌 건수의 비율

$$\frac{True\ Positive + True\ Negative}{True\ Positive + True\ Negative + False\ Positive + False\ Negative}$$

재현율: 실제로 정답이 1이라고 할 때 모델도 1로 예측한 비율

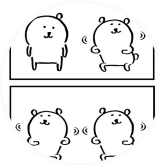
$$\frac{True\ Positive}{True\ Positive + False\ Negative}$$

정밀도: 모델이 1이라고 예측한 것 중에서 실제로 정답이 1인 비율

$$\frac{True\ Positive}{True\ Positive + False\ Positive}$$

F1-스코어: 정밀도가 높으면 재현율이 낮고, 재현율이 높으면 정밀도가 낮다. 이런 문제를 해결하기 위해 이용하는 것이 F1-스코어.

$$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$



카사바

안녕하세요 :)

0개의 댓글

댓글을 작성하세요

댓글 작성

Powered by Stellate